Reproducing Minutiae Clustering in Fingerprint Spoof Buster Lite, Without the Paper's Data

Muzi [TODO: surname] ([TODO: student number]), University of the Witwatersrand
Repository: `https://github.com/TODO/spoofbuster-lite-repro`

October 28, 2026

Abstract

Chugh and Jain [1] report that classifying patches at 10 k-means centroids of the minutiae, instead of at every minutia, keeps fingerprint presentation-attack detection at TDR = 95.3% (against 95.7%) at FDR = 0.2%, while cutting phone inference time from 1,554 ms to 302 ms. The paper's data is gated by a Michigan State University agreement, and I could not obtain it, so I reimplemented the full pipeline and tested the claim's mechanism on seeded synthetic fingerprints. With 10 centroid patches, TDR changed by -0.1 points from the all-minutiae baseline, within my one-point criterion. A second training seed, however, gave -3.2, and the baseline's sampling error is 4.1 points, so the effect is below what I can resolve. Clustering cut CNN time 3.9× but total latency only 1.6×, because minutiae extraction dominates. I also show that the paper's Table 2 is internally inconsistent.

1 The claim

Fingerprint Spoof Buster [2] detects presentation attacks (spoofs) by cutting a 96×96 patch around every minutia, aligned to the minutia direction, scoring each patch with MobileNet-v1 [3] and averaging the scores. Its cost grows with the number of minutiae, 35 per image in the paper's data. Section 3 of [1] proposes Spoof Buster Lite: cluster the minutia coordinates with k-means, cut one patch at each centroid, and average the patch scores weighted by cluster size.

The claim I set out to reproduce is in Table 2 of [1]: *with $k = 10$ clusters, TDR at FDR = 0.2% falls only from 95.7% to 95.3%, while inference time falls from 1,554 ms to 302 ms.* TDR is the fraction of spoofs detected, and FDR the fraction of bona fide prints wrongly flagged. The support is that one table: six values of k and the baseline, each given as mean ± s.d. over "5-fold cross-validation", with times from a quantised model on a Redmi Note 4. The data are 5,743 bona fide and 4,912 spoof images of 12 materials (MSU-FPAD and PBSKD).

My research question, fixed in the proposal: *does $k = 10$ keep TDR within one percentage point of the all-minutiae baseline while reducing the number of MobileNet evaluations?*

2 Route and setup

Data I could not get. [TODO: In your words, with dates: requested access; Tarang Chugh replied on 26 Oct 2026 that he only had access as a PhD student, no longer has it and may not share it, and pointed you to the lab; you emailed the MSU lab on [date]; no reply by the deadline.] I searched for other legitimate sources. The closest substitute, LivDet 2015 [4], includes a CrossMatch subset but also requires a signed licence. I did not use unofficial copies.

Synthetic stand-in. So that the method could still be tested end to end, I generated a seeded dataset of 1,600 bona fide prints (800 for testing) and 6 artefact "materials" of 200 prints each, all 288×288 pixels. Ridges are grown by Gabor-filtering noise along

a core–delta orientation field, as in SFinGe. Every image gets random contrast, blur, noise and dry or wet blotches. Each material adds one artefact: thick ridges, blur, grey-level quantisation, moisture, ridge breaks, or elastic distortion. The artefact has a random strength and covers 40–100% of the print, so where a patch lands matters. These are loose analogies, not models of silicone or gelatin, so *no absolute number here says anything about real spoof detection.* The stand-in tests the mechanism: does scoring k centroid patches instead of every minutia patch lose accuracy, and how does cost scale?

Reimplementation. The authors' code is not public, so all code is mine, written with AI assistance and checked line by line. I used library code only for the MobileNet-v1 architecture (Keras), k-means (scikit-learn) and basic image operations. `bash run_all.sh --synthetic` regenerates every number and figure from a clean checkout, and the same pipeline runs on real data by pointing it at a folder.

Minutiae. The paper does not name its extractor. Mine uses local normalisation, a structure-tensor orientation field, Gabor enhancement, skeletonisation and the crossing number, then removes points near the print edge and pairs closer than one ridge period. It finds 39.1 minutiae per image, close to the paper's 35, so the all-minutiae baseline costs 39.1 CNN calls.

Centroid alignment. A centroid is not a minutia and has no direction. The paper does not say how centroid patches were rotated, so I align each one to the ridge orientation field at the centroid (defined only modulo 180°).

Classifier. I use MobileNet-v1 ($\alpha = 1$) with a 96×96 input, trained on minutia-centred patches that inherit their image's label. The final feature map is 3×3, so global average pooling matches the paper's 3×3 pool. I used Keras because TF-Slim needs TensorFlow 1.x. The paper gives no training recipe. On a 2-core CPU without ImageNet weights, I trained from scratch: Adam with learning rate 10^{-3}, batch 128, 5 epochs, class-balanced loss, dropout 0.2, and at most 12 patches per image. For the first epochs, validation AUC sat at exactly 0.5 while training AUC rose: the BatchNorm running statistics lagged the weights, so inference outputs were constant. As a final training step I recompute those statistics with the final weights, without changing any weights. This raised seed-0 validation patch AUC from 0.813 to 0.842.

Table 1: Table 2 of [1] next to my synthetic-data reproduction. Paper: phone, quantised model, s.d. over 5 repeats. Ours: CNN calls per image, latency on one CPU thread (TFLite), TDR (%) at FDR = 0.2% with s.d. over 5 k-means seeds (bootstrap s.d. for "All"), and TDR at FDR = 1%. Only differences between rows are comparable across the two halves.

	Paper (Redmi Note 4)		Ours			
k	ms	TDR@0.2	CNN calls	ms (CPU)	TDR@0.2	TDR@1
5	148	93.9 ± 1.1	5.0	86	33.9 ± 3.2	41.9
10	302	95.3 ± 0.5	10.0	97	35.3 ± 1.4	44.9
15	447	95.3 ± 0.5	15.0	108	35.5 ± 1.1	45.4
20	607	95.3 ± 0.6	20.0	119	36.8 ± 1.2	45.9
All	1554	95.7 ± 0.1	39.1	156	35.4 ± 4.1	45.4

Protocol. One trained model scores every setting, as in the paper. For each $k \in \{5, 10, 15, 20\}$, k-means is repeated with 5 seeds ($n_{\text{init}} = 1$). The threshold is set per setting so that at most 0.2% of bona fide test images score above it. I also report TDR at FDR = 1% and a bootstrap s.d. For timing, the model runs as 8-bit TFLite on one CPU thread, one patch at a time like the phone app, on 100 test images. Only the scaling with k is comparable to the paper.

3 Results next to the paper

Because the data differ, the like-for-like quantity is the *change* from the all-minutiae baseline (Table 1, Fig. 1). My baseline is 35.4% (bootstrap s.d. 4.1). With 10 centroid patches it is 35.3 ± 1.4%, a change of -0.1 points against the paper's −0.4. The shape matches the paper. Both curves dip at $k = 5$ (mine -1.5, the paper's −1.8) and are flat from $k = 10$ upwards, so on this point the claim reproduces: about 10 centroid patches carry as much information for the classifier as all 39.1 minutia patches. At FDR = 1%, where the threshold is less noisy, the change at $k = 10$ is -0.5 points.

The absolute level does not reproduce, and was never expected to: 35.4% against 95.7%. My detector

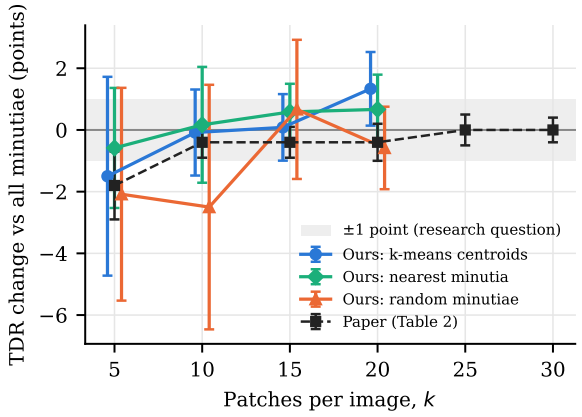


Figure 1: Each curve is the change in TDR at FDR = 0.2% from its own all-minutiae baseline, because the absolute levels come from different data. Error bars: s.d. over 5 k-means or random seeds; ours are offset slightly in k for legibility. Grey band: the ±1-point criterion of my research question.

Table 2: TDR (%) per synthetic material at the all-minutiae FDR = 0.2% threshold, model seed 0; k = 10 pooled over 5 k-means seeds.

Material	n	All	k=10
blurred	40	72.5	75.0
broken ridges	40	2.5	4.5
distorted	40	7.5	7.0
moist	40	22.5	25.0
printed	40	12.5	15.0
thick ridges	40	95.0	95.0

Table 3: Patch-selection ablations, TDR (%) at FDR = 0.2%, mean ± s.d. over 5 seeds. "Centroid": the paper's method. "Unweighted": plain mean of the centroid scores. "Nearest minutia": same clusters and weights, patch centred on the minutia closest to each centroid. "Random": k random minutiae.

k	centroid (paper)	unweighted	nearest minutia	random
5	33.9 ± 3.2	34.1 ± 3.5	34.8 ± 1.9	33.3 ± 3.4
10	35.3 ± 1.4	36.8 ± 1.1	35.6 ± 1.9	32.9 ± 4.0
15	35.5 ± 1.1	37.3 ± 1.4	36.0 ± 0.9	36.1 ± 2.3
20	36.8 ± 1.2	38.5 ± 1.5	36.1 ± 1.1	34.8 ± 1.3
All	35.4 ± 4.1 (bootstrap s.d.)			

is weak. It was trained from scratch for 5 epochs on 12 patches per image, and two of my synthetic materials are almost undetectable (Table 2). "No loss" at 35% TDR is weaker evidence than "no loss" at 95%, because a detector near its ceiling has less room to lose.

4 Numbers the paper does not report

Clustering, or just fewer patches? The paper never compares k-means with simply classifying k random minutiae. At $k = 10$, random minutiae give 32.9 ± 4.0%, against 35.3 ± 1.4% for centroids (Table 3). The means are within noise of each other, but random selection varies about three times as much between seeds at $k = 10$. Here, clustering seems to buy stability rather than accuracy: centroids spread the patches over the whole print, while a random draw can leave out the region that carries the artefact. At $k = 15$ and 20, the two methods coincide.

Does the size weighting matter? Without weights, 10 centroids give 36.8 ± 1.1%, no worse than the weighted rule. The paper motivates the weighting but never tests it. On my data it does not help.

A second training seed. A model retrained with a different seed reaches 37.9% with all minutiae (Table 4), against 35.4% for seed 0: retraining alone moves the baseline by more than the paper's whole $k = 10$ effect. For seed 1, $k = 10$ scores 34.8 ± 2.6%, a change of -3.2 points, which *fails* the one-point criterion. At FDR = 1%, the seed-1 change is -1.4. My answer to the research question therefore depends on the training seed. The honest conclusion is that the effect of clustering is smaller than the noise my setup can resolve, not that it is zero.

One threshold for every setting. The paper recomputes the threshold for each k, but a deployed system would keep one. Reusing the all-minutiae threshold at $k = 10$ gives TDR = 36.9% at an actual FDR of 0.25%, compared with the 0.2% target. Cen-

Table 4: Two training seeds. "Fixed thr.": the all-minutiae threshold reused at $k = 10$ instead of recomputed, as stated in my proposal.

Model seed	All, TDR@0.2	k=10, TDR@0.2	k=10, fixed thr. TDR / FDR
0	35.4 ± 4.1	35.3 ± 1.4	36.9 / 0.25
1	37.9 ± 7.1	34.8 ± 2.6	38.6 / 0.30

Table 5: My per-image latency (ms) by stage: one CPU thread, TFLite with 8-bit weights, 100 test images. Minutiae extraction is the same for every k.

k	patches	minutiae	select	cut	CNN	total
All	39.6	72	0.0	1.8	82	156
5	5.0	72	2.9	0.3	11	86
10	10.0	72	3.2	0.5	21	97
15	15.0	72	3.9	0.7	32	108
20	20.0	72	4.5	0.9	42	119

troid patches shift the bona fide scores up slightly, so a device switching to $k = 10$ would have to recalibrate its threshold. The paper's per-setting thresholds hide this.

5 What did not reproduce

5.1 The data, and therefore the absolute numbers

The largest gap is the one I could not close. Without MSU-FPAD and PBSKD I cannot test whether 95.3% and 95.7% hold, or whether the result holds on real materials such as silicone, gelatin and Play-Doh, which fool CNNs in ways my artefacts do not. My synthetic results test only whether centroid patches are enough for a patch classifier *in principle*. Everything in this section that does not depend on the data, namely the latency analysis and the paper's internal consistency, I could check fully.

5.2 The speed-up

Cutting from 39.1 patches to 10 makes the CNN stage 3.9× faster, roughly in proportion to the patch count. The whole pipeline, however, goes from 156 ms to

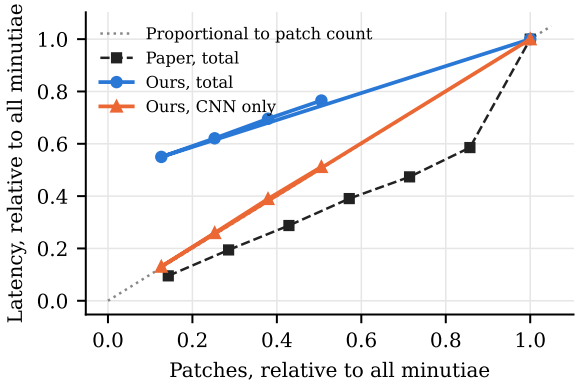


Figure 2: Latency against patch count, both relative to all minutiae. The dotted line is latency proportional to patch count. The paper's points fall below it. My CNN stage follows it, and my total lies above it because minutiae extraction is a fixed cost.

97 ms, only 1.6× faster (Table 5). Minutiae extraction takes 72 ms per image whatever k is: clustering needs the minutiae first, so it can only shrink the CNN stage. The paper does not say whether its 302 ms includes minutiae extraction. If it does, the phone must extract minutiae far faster, relative to one CNN call, than my extractor does.

5.3 Table 2 is not internally consistent

1. **Latency per patch.** For $k = 5$ to 30, Table 2 implies about 30 ms per patch (148/5, 302/10, 910/30), which matches the 30 ms per quantised patch given in Section 3. At that rate all 35 minutiae would take about 1,050 ms, not 1,554 ms, so the baseline row carries about 500 ms that the clustered rows do not. Fig. 2 shows this: the paper's points fall *below* the proportional line, which cannot happen if cost scales only with the number of patches. The claimed 5.1× speed-up (1,554/302) exceeds the 3.5× cut in patches (35/10). The paper itself says "approximately 4-fold" in one place and "almost 80%" in another.

2. **The baseline TDR.** Section 2.4 gives

TDR = 97.20% at FDR = 0.2% with all materials known in training. Table 2's all-minutiae row, also with all materials known, is 95.7%. The paper does not explain the gap: the 96×96 retraining, quantisation or a different split could each cause it.

3. **What the s.d. measures.** The all-minutiae row has s.d. 0.1 although it involves no k-means at all. Its variation must come from something other than initialisation, which suggests the "5 folds" are five data splits. If so, the ± 0.5 at $k = 10$ mixes split noise with initialisation noise, and my s.d. over k-means seeds on one split is not the same quantity.

5.4 The operating point is fragile

At FDR = 0.2% with 1,000 bona fide images, the threshold is the third-highest bona fide score, so one or two unusual live prints decide the result. With my 800 bona fide test images it is the second-highest score, and my bootstrap s.d. for the baseline is 4.1 points (seed 0) and 7.1 (seed 1). My detector is weaker, so its error is larger than the paper's would be. But even the binomial error of a TDR near 95%, measured on about 1,000 spoofs, is roughly 0.7 points: larger than the paper's 0.4-point gap, and far larger than its ± 0.1. The paper does not state its spoof test count, so this is an estimate. Either way, Table 2 alone cannot show that $k = 10$ and all minutiae perform the same.

5.5 What I tried

The model is trained on minutia-centred patches but tested on centroid-centred ones, and a centroid often lies between minutiae. To test whether this mismatch costs accuracy, I moved each patch to the minutia nearest its centroid, keeping clusters and weights. This gave $35.6 \pm 1.9\%$ at $k = 10$ (Table 3), the same as centroids within noise. The centroid–minutia mismatch is therefore not what limits accuracy here. Two other problems had to be fixed along the way. The BatchNorm recalibration described in Section II was needed before the from-scratch models gave non-constant outputs. And the first run of the second seed was killed for lack of memory when I ran a test alongside it on the same 8 GB machine, so I reran it alone.

5.6 What I still cannot explain

First, why the second seed shows a three-point drop at $k = 10$ when the first shows none. With two seeds I cannot tell whether this is a real seed effect or threshold noise; it would take many more seeds than my CPU budget allowed. Second, the roughly 500 ms of the paper's baseline latency that per-patch cost does not account for. Third, the paper's own 97.2% versus 95.7%. And fourth, the question my synthetic data cannot reach: whether the flat curve holds for real materials, where a spoof can look bona fide in one region and fake in another.

6 Data ethics

The paper's datasets contain fingerprints of real people, captured under MSU's consent and released only to researchers who sign a data agreement. That gating is why I could not obtain them in time, and it is also why I did not use any unofficial copy: doing so would breach the terms under which the subjects' data was shared. All results here use synthetic fingerprints, so the repository contains no one's biometric data. When run on real data, the code keeps images, patches and minutiae out of version control, because minutiae coordinates are a biometric template. Presentation-attack research is also dual-use: per-material results can show an attacker which materials evade a detector.

7 Conclusion

The paper's data were out of reach, so I could reproduce the mechanism but not the numbers. On synthetic fingerprints, the shape of Table 2 reproduces: TDR dips at $k = 5$ and is flat from $k = 10$. The size of the effect is below what my setup can resolve: retraining and the 0.2% threshold each move

TDR by more than the paper's 0.4 points. The fourfold CNN saving reproduces. The fivefold end-to-end speed-up does not: minutiae extraction, which clustering cannot remove, keeps the real gain at 1.6×, and the paper's own latencies imply a baseline about 500 ms heavier than its per-patch cost explains. The number I would defend first is the stage breakdown in Table 5, because it does not depend on the data I could not obtain.

Use of AI tools

An AI assistant (Claude) wrote the code to my specification and drafted this report. At my request, it ran the experiments in its cloud workspace. I checked the code, the numbers and the interpretation, and I am responsible for all of them. [TODO: adjust so it states exactly what you did and what you rewrote]

References

[1] T. Chugh and A. K. Jain, "Fingerprint presentation attack detection: Generalization and efficiency," in *Proc. ICB*, 2019. arXiv:1812.11574.

[2] T. Chugh, K. Cao, and A. K. Jain, "Fingerprint Spoof Buster: Use of minutiae-centered patches," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 9, pp. 2190–2202, 2018.

[3] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[4] D. Yambay *et al.*, "Review of the fingerprint liveness detection (LivDet) competition series: from 2009 to 2021," arXiv:2202.07259, 2022.